TEAM #46

Hublet – APLS

Architecting a Scalable Market Intelligence System

Team Rachit Mehta · Arijeet Paul · Rakshit Agarwal · Jenik Gajera · Abhinav PV

Client: Vaibhav Singh (IIT Delhi) | Mentor Email: vaibhavsinghiitd28@gmail.com|Jan – Mar 2026 |TA Guide: Rudra Chaudhary

Evolutionary Prototyping | Agile Sprints | SOLID Principles

Real Estate Lead Matching · Backend Architecture · DevOps · AI/NLP

INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY
HYDERABAD

AGENDA | Presentation Roadmap

01 **Problem Statement**
Core challenge and real-world pain points

02 **Client Requirements**
What Vaibhav Singh wanted — from all MoMs

03 **Development Model**
Evolutionary Prototyping — why it fits

04 **What We Built – R1**
Milestones, demos, and deliverables

05 **What We Built – R2**
Ongoing + upcoming milestones

06 **System Architecture**
Layered design and tech stack

07 **SE & Design Principles**
SOLID, GRASP, Design Patterns applied

08 **TDD & Testing Strategy**
Test-Driven Development cycle in Hublet

09 **Functional Requirements**
16 use cases across all actors

10 **Non-Functional Requirements**
8 NFRs with measurable targets

11 **Scope Creep Management**
MoSCoW, feature freeze, change control

12 **Risk Register**
Identified risks + active mitigations

13 **Weekly Progress Summary**
Status tracker — 7 weeks

14 **Conclusion & Next Steps**
Takeaways and remaining roadmap

Problem Statement & Client Requirements

Understanding what we were asked to build

INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY
HYDERABAD

01 | Problem Statement

"Real estate marketplaces generate large volumes of heterogeneous listings and buyer enquiries that are impossible to match efficiently through manual processes — leading to missed opportunities, slow response times and poor conversion rates."

✗ Manual Matching

Brokers manually connect buyers and sellers — slow, error-prone, unscalable

✗ Unstructured Buyer Intent

Preferences expressed in natural language: no standard structured form

✗ No Lead Lifecycle

Zero visibility from initial enquiry to deal closure — no state tracking

✗ Heterogeneous Data

Property listings scattered across MagicBricks, 99acres, WhatsApp with no unified schema

✗ No Price Intelligence

No mechanism to quote or benchmark property prices against market data

✗ Contact Verification

Agents and sellers unverified — false leads waste buyer time

02 | Client Vision — The 'Black Box' Matching Engine

> *"Hublet is NOT a marketplace. Marketplaces like MagicBricks and 99acres already exist. We want a black-box (or glass-box) matching engine that predicts, scores, and routes leads."*
>
> — *Vaibhav Singh*

Core Engine

- Ingest property data from web portals & WhatsApp
- Parse buyer intent from natural language
- Score matches via deterministic weighted algorithm
- Lead lifecycle state machine (NEW → CLOSED)
- Audit log for every event in the system

Data Strategy

- Primary: MagicBricks web scraping (6-hour cycle)
- Secondary: Facebook Real estate groups (LLM-structured)
- Commercial RE focus — Bombay, Delhi, Hyderabad, Pune
- Authenticated scraping for seller contact details

Infrastructure

- Cloud DB: MongoDB Atlas
- RESTful API with role-specific dashboards
- 24-hour scrape cycle acceptable for demo stage
- Keyword-based NLP (No heavy models used yet.)
- LLM API (Claude / Groq) for unstructured post parsing

03 | Requirements Evolution: Week by Week Timeline

Week 1
Jan 18 – Jan 25

I. Establish high-level scope

II. Pipeline & data ingestion ambiguities identified

III. Score-matching & CMA concepts introduced

IV. Frontend vs backend clarity needed

V. Authentication type to be decided

Week 2
Jan 26 – Feb 1

I. Async review of Hublet-ALPS legacy codebase

II. NLP keyword extraction prototype built

III. Intent parsing: 'Looking for X in Y' pattern

IV. Facebook Scraping planned

Week 3
Feb 2 - Feb 9

I. NOT a marketplace — differentiation confirmed

II. Exa AI for web extraction suggested

III. Photo verification for seller identity

IV. Location auto-fill on map pin feature

Week 4
Feb 10 - Feb 17

I. Tried to integrate Supabase.

II. Improved Scraping.

III. Discussed intent matching

Week 5
Feb 17 – Feb 24

I. Supabase Integration failed, explored alternatives

II. Team discussed agenda for next week

Week 6
Feb24- Mar 3

- Live scraping from MagicBricks demonstrated
- Match threshold raised to 70%
- MongoDB Atlas as Supabase fallback
- LLM (Claude/Groq) for Facebook posts

Requirements were continuously refined across 6 weeks — confirming following the Evolutionary Prototyping model of the project.

Matching Algorithm

Rule-Based Weighted Scoring — Design, Formula & Implementation

ALGO | Matching Algorithm — High-Level Overview

Rule-Based · Deterministic · Weighted Scoring · NOT machine learning · Seller rating/trustScore excluded

totalScore =	0.35	× location Score	(35%)
+	0.30	× budget Score	(30%)
+	0.20	× size Score	(20%)
+	0.15	× amenities Score	(15%)

Rounded to 2 decimal places

35%	30%	20%	15%
Location	Budget	Size	Amenities

1. Load Buyer Intent
2. Fetch Active Properties
3. Score Each Property
4. Filter (minScore≥50)
5. Sort Descending
6. Persist + Return (limit 50)

ALGO | Component Scores — Location & Budget

Location Score (weight: 35%)

Exact locality match	100
Partial text containment match	70
No match	0

Note: Matching is string-based — substring check.
Geospatial/semantic similarity is a known limitation.

Budget Score (weight: 30%)

No budget preference set Neutral — buyer didn't specify	50
Property inside buyer range Perfect budget fit	100
Above max budget ≤10% Slightly over — still acceptable	70
Above max budget ≤20% Noticeably over budget	40
Above max budget >20% Too expensive	10
Below min budget ≤10% Slightly under — near range	70
Below min budget ≤20% Under buyer's range	40
Below min budget >20% Much too cheap — mismatch	10

Fallback (unexpected case): 30
Prices in Indian Rupees (Crores); same unit for buyer + property.

ALGO | Component Scores — Size & Amenities

Size Score (weight: 20%)

Size Score = BHK sub-score + Area sub-score (capped at 100)

BHK Sub-score (max 50 pts)

Exact BHK match	+50
BHK difference = 1	+25
No BHK preference set	+25

Area Sub-score (max 50 pts)

Area within buyer min–max range	+50
Closest diff < 200 sq ft	+25
Closest diff < 500 sq ft	+10
No area preference set	+25

Amenities Score (weight: 15%)

```
if (no amenity preference):
  amenitiesScore = 50   // neutral
else:
  amenitiesScore = round( matched / requested × 100 )
```

Worked Example

Buyer requests: "gym, parking, pool" (3 amenities)

Property has: "gym, parking" (2 match)

amenitiesScore = round(2 / 3 × 100) = 67

Known Limitations

- Requires near-exact text normalization
- 'gym' ≠ 'Gym' ≠ 'gymnasium' — no fuzzy match yet
- Seller trust/rating NOT included in score

Development Model

Evolutionary Prototyping — the right fit for Development of Hublet

INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY
HYDERABAD

04 | Why Evolutionary Prototyping

- Working prototype delivered every week for client feedback and improvement made accordingly.
- Requirements emerged gradually — NLP, matching, scraping refined iteratively.
- Each prototype led to the production system.
- Client could validate the site behavior with real demo data each session
- Architecture evolved: SQLite → Supabase → MongoDB Atlas as needs changed
- Stretch goals (ML, LLM) isolated and deferred without breaking the core
- Risk reduced: each sprint had a shippable increment

What We Have Built

R1 Milestones, R2 Roadmap & System Architecture

INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY
HYDERABAD

05 | Release 1 — Core Backend System (Milestones 1–12)

M1–M3

Setup & Architecture

Done

Jan 25–30

Team formation, tech stack finalization, GitHub repo + dev environment initialized

M4–M5

Database & CRUD APIs

Done

Feb 1–2

Prisma ORM schema design, SQLite config, full CRUD for Buyer/Seller/Property entities

M6

Intent Parsing Engine

Done

Feb 4

Keyword NLP: regex extracts BHK, locality, budget range (lakh/crore), area, amenities from free text

M7–M8

Dashboards & Scraper

Done

Feb 8–10

Admin/Buyer/Seller dashboards in React+Vite; MagicBricks scraping pipeline + 6-hour cron scheduler

M9–M10

Matching Engine + Lead FSM

Done

Feb 12

Rule-based weighted scoring (Location 35%, Budget 30%, Size 20%, Amenities 15%), Lead FSM (NEW→ENRICHED→QUALIFIED→NOTIFIED→CONTACTED→CLOSED)

M11–M12

JWT Auth + R1 Deliverable

Done

Feb 13–14

JWT authentication, protected route middleware; R1 Requirements Document formally submitted

06 | System Architecture — Layered Design

Presentation Layer	**React 18 + TypeScript + Vite + Axios** Admin Dashboard · Buyer Dashboard · Seller Dashboard (role-specific views)
API Layer	**Express.js 4.18 + JWT Middleware + CORS** RESTful Routes → Request Validation (Zod) → Controllers → Services
Business Logic Layer	**Rule Based Matcher + Keyword Intent Parser + Lead State Machine** Weighted Scoring · NLP Parsing · FSM Transitions · WorkflowEvent Logging
Data Access Layer	**Prisma ORM + SQLite / MongoDB Atlas** Type-safe queries · Prisma Migrations · FK constraints · Cascading deletes

Scraper Module
Python · Selenium and Appify · 6hr cron · MagicBricks

External Data
· MagicBricks · 99acres · Facebook Groups

LLM APIs
Claude API · Groq · Unstructured post parsing

DevOps
GitHub Actions · Docker · Prisma Studio

SE & Design Principles

SOLID, GRASP, Design Patterns, Industry Practices

INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY
HYDERABAD

07 | SOLID Principles — Applied in Hublet

S

Single Responsibility Principle

A class/module should have only one reason to change.

In Hublet: Rule Based Matcher only scores. Keyword Intent Parser only parses. Lead State Machine only validates transitions. No class does more than one job.

O

Open / Closed Principle

Open for extension, closed for modification.

In Hublet: Matcher interface allows swapping RuleBasedMatcher for an ML Matcher without touching controllers or routes. IntentParser is also pluggable.

L

Liskov Substitution Principle

Subtypes must be substitutable for their base types.

In Hublet: Any future Matcher implementation (e.g. CollaborativeFilterMatcher) can replace RuleBasedMatcher transparently — no calling code changes.

I

Interface Segregation Principle

Clients should not depend on interfaces they don't use.

In Hublet: Buyer API, Seller API, Admin API are separate routers. No route imports handlers it doesn't need — thin, focused interfaces.

D

Dependency Inversion Principle

High-level modules should not depend on low-level modules.

In Hublet: Controllers depend on Service interfaces, not on Prisma directly. DB layer is abstracted via Prisma ORM — swapping DB requires no controller changes.

08 | Industry Engineering Practices

Version Control

- Git feature-branch workflow (feature/ → main)
- Pull request reviews before any merge
- Annotated git tags for each release snapshot (submission-week-*)
- Commit history as immutable audit trail
- Weekly snapshot tags protected — students cannot force-push

CI / CD Pipeline

- GitHub Actions: automated workflow on every push
- Weekly snapshot workflow creates release + integrity tag
- Required files enforced: StatusTracker.xls + ProjectPlan.md
- Tampering detection: release body must match annotated tag
- Docker Compose for consistency

Code Quality

- TypeScript strict mode — compile-time type safety
- Zod schema validation at every API boundary
- ESLint configured for consistent code style
- Layered architecture prevents spaghetti dependencies
- Prisma type-safe ORM eliminates raw SQL mistakes

Security Engineering

- JWT authentication on all protected routes
- Role-Based Access Control — Admin / Buyer / Seller scopes
- CORS middleware restricts cross-origin requests
- Input sanitization prevents injection attacks
- Security audit planned as M23 before final demo

Documentation

- Meeting Minutes (MoM) after every client sync
- StatusTracker.xls updated every sprint
- Markdown technical docs committed to repo
- Swagger / OpenAPI specification planned (M15)
- README with setup, architecture, and API guide

Observability

- WorkflowEvent table logs every CUD + state change
- Timestamped records with event type + metadata
- Admin dashboard shows complete audit trail by timestamp
- Monitoring and logging infrastructure planned (M21)
- Error states and invalid transitions also logged

Functional & Non-Functional Requirements

What the system must do and how well it must do it

INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY
HYDERABAD

09 | Functional Requirements — All 16 User Cases

UC ID	Use Case Name	Actor	Rel	Description
UC-01	Buyer Registration	Buyer	R1	New buyer creates account with preferences (localities, budget, BHK, amenities, notes)
UC-02	Buyer Login	Buyer	R1	Existing buyer authenticates via email; redirected to dashboard
UC-03	Update Buyer Preferences	Buyer	R1	Buyer modifies search params; system triggers re-matching automatically
UC-04	Seller Registration	Seller	R1	New seller creates account with name, email, phone, seller type
UC-05	Seller Login	Seller	R1	Existing seller authenticates via email; redirected to seller dashboard
UC-06	Add Property Listing	Seller	R1	Seller adds property with title, locality, price (Crores), BHK, area, amenities
UC-07	View Seller Properties	Seller	R1	Seller views all listed properties with status and details
UC-08	Find Buyer–Property Matches	System	R1	Weighted scoring: Location 35%, Budget 30%, Size 20%, Amenities 15%
UC-09	View Match Results (Buyer)	Buyer	R1	Buyer views ranked properties with score breakdown + seller contact
UC-10	View Matched Buyers (Seller)	Seller	R1	Seller views buyers matched to each listing with match scores
UC-11	Lead Creation	System	R1	System creates Lead entity linking buyer + property; initial state = NEW
UC-12	Lead State Transition	Admin/System	R1	NEW→ENRICHED→QUALIFIED→NOTIFIED →CONTACTED→CLOSED
UC-13	Admin Dashboard	Admin	R1	Tabbed view: Buyers, Sellers, Properties, Leads, Matches, Logs + stats
UC-14	Audit Trail Logging	System	R1	Every CUD + state change logs WorkflowEvent with timestamp + metadata
UC-15	Property Data Scraping	System	R2	Scheduled 6-hour scrape from MagicBricks; normalized into DB schema
UC-16	NLP Intent Parsing	System	R1	Regex keyword extraction from free text → ParsedIntent object

10 | Non-Functional Requirements (NFRs)

NFR-1 **Performance** — < 2 sec

Match algorithm scores up to 1,000 properties per query on standard dev hardware

NFR-2 **Scalability** — Pluggable

Matcher & Intent Parser interfaces allow algorithm swap without modifying any calling code

NFR-3 **Reliability** — 100% FSM

All lead transitions validated by state machine; invalid transitions rejected + logged pre-DB

NFR-4 **Auditability** — Full log

Every CUD + state change generates WorkflowEvent with timestamp, event type, and metadata

NFR-5 **Portability** — 1-line swap

SQLite/ MySQL requires only a Prisma datasource provider config change

NFR-6 **Maintainability** — Type-safe

Layered architecture + TypeScript strict mode prevents runtime type errors and coupling

NFR-7 **Usability** — 3 roles

Role-specific dashboards with visual match scores, colour-coded status, responsive layout

NFR-8 **Data Integrity** — Constrained

FK cascading deletes; unique constraints on emails and buyer-property match pairs in schema

Scope Management & Risk Mitigation

How we kept the project on track

INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY
HYDERABAD

11 | Scope Creep Prevention — MoSCoW + Change Control

MoSCoW Prioritization

MUST have

- Buyer/Seller registration & login
- Property CRUD APIs
- Rule-based weighted matching engine
- Lead lifecycle state machine
- Admin dashboard + audit log
- JWT authentication + RBAC

SHOULD have

- MagicBricks web scraper
- 99 acres web scraper
- Keyword NLP intent parser
- Docker containerization
- Unit & integration tests (60%+)

COULD have

- Email notification system
- BullMQ async job queues
- Analytics dashboard
- Advanced deduplication
- CI/CD production pipeline
- Facebook LLM parsing (stretch)

WON'T (this release)

- ML-based property valuation
- Collaborative filtering
- A/B testing ML vs rules
- Flask/FastAPI microservice

Feature Freeze pr Release

Weekly Progress

Status tracker — 7 sprints of Evolutionary Development

INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY
HYDERABAD

12 | Weekly Sprint Summary — StatusTracker (Weeks 1–7)

11	2	11	7	120
Milestones Done	In Progress	Pending (R2)	Sprints Elapsed	Team Hours Est.

Week	Key Activities	Hours	Status
Week 1 **Jan 18–25**	Fix client meeting, GitHub setup, repo init, role assignment, project synopsis, status tracker	9 hrs	Done
Week 2 **Jan 26–Feb 1**	Async MoM review, NLP prototype built, questions for requirements, module breakdown, communication setup	16 hrs	Done
Week 3 **Feb 2–9**	Backend dev + mock data, Supabase DB design, client meeting, feedback on NLP, Supabase credentials requested	28 hrs	Done
Week 4 **Feb 10–17**	Improved scraping pipeline, intent-matching algorithm discussions, Supabase credentials received	11 hrs	Done
Week 5 **Feb 17–24**	Supabase integration (delayed), team sync meeting, architecture review	5 hrs	Done
Week 6 **Feb 24–Mar 3**	Supabase integration live, intent parsing + rule matching implemented, test doc created, client MoM planned	16 hrs	Done
Week 7 **Mar 3–9 (NOW)**	Client meeting (Mar 7 MoM), Facebook scraping, MongoDB integration supabase failed, deployment, code refactoring	10 hrs	Done

Towards R2

Status tracker — 7 sprints of Evolutionary Development

INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY
HYDERABAD

13 | Release 2 — Scalability, Security & Deployment

M13–14

Validation & RBAC

Ongoing

Feb 20–22

Zod input validation, scraped data normalization, Role-Based Access Control

M15–16

API Docs & Tests

Pending

Feb 23–25

Swagger/OpenAPI documentation; unit + integration tests targeting 60%+ coverage

M17–18

Docker & BullMQ

Pending

Feb 27–Mar 3

Docker Compose containerization, BullMQ async job queue for scraping workers

M19–20

Notifications & Analytics

Pending

Mar 5–8

Email notification system, advanced filtering, deduplication, analytics dashboard

M21–22

Monitoring & CI/CD

Pending

Mar 12–18

Logging, performance monitoring, CI/CD pipeline and production deployment config

M23–24

Final QA & Demo

Pending

Mar 22–25

Security audit, system integration testing, final demo + deployment documentation

ALGO | Intent Parsing — KeywordIntentParser (intent-parser.ts)

Input: "Looking for a 3BHK apartment in Bandra or Juhu under 1.5 crore with gym and parking"

Field	Regex / Heuristic Pattern	Extracted Value
`localities`	keyword "in" followed by place names; also comma-split	`["Bandra", "Juhu"]`
`BHK`	pattern: /\d+\s*(bhk|BHK|bedroom)/	`"3"`
`budgetMax`	patterns: "under X", "below X", "within X crore/lakh"	`"150" (Lakhs)`
`budgetMin`	"above X", "minimum X", "at least X"	`null (not specified)`
`area`	patterns: "X sqft", "X sq.ft", "X square feet"	`null (not specified)`
`amenities`	keyword list match: gym, parking, pool, lift, etc.	`["gym", "parking"]`

Output: ParsedIntent

```
{ localities: ["Bandra","Juhu"], bhk: 3, budgetMax: 150, budgetMin: null, areaMin: null, areaMax: null,
amenities: ["gym","parking"] }
```

Thank You For Your Time!

Questions ?

INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY
HYDERABAD